

Lecture 10

Combinational Analysis and Design



Combinational circuits · the analysis procedure · the design procedure · code converters · seven-segment decoding

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 4, §4.1–4.4

Today's roadmap

1

§4.1–4.2

Combinational

Outputs depend on inputs only

2

§4.3

Analysis

Circuit → functions → table

3

§4.4

Design

Words → table → circuit

4

§4.4

Code converters

Shared terms; don't-cares

By the end of class you can...

- Tell a combinational circuit from a sequential one
- Analyze a gate diagram to Boolean functions and a truth table
- Follow the design procedure from a word problem
- Design multiple-output circuits that share gates
- Use unused input codes as don't-cares

Agenda for today's session

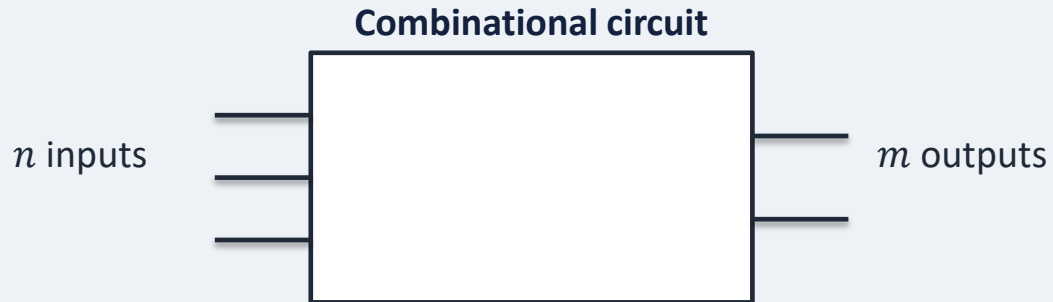


Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap	—
0:05 – 0:15	Combinational versus sequential	—
0:15 – 0:40	The analysis procedure	Example 1 · Your turn 1
0:40 – 0:50	Break	
0:50 – 1:10	The design procedure	Example 2
1:10 – 1:40	Code converters and display decoders	Examples 3, 4 · Your turn 2
1:40 – 1:50	Concept check and wrap-up	Exam 1 reminder

Combinational versus sequential

§4.1–4.2

Combinational



Sequential



Combinational

Outputs depend only on the present inputs.

Described by a truth table or Boolean functions: 2^n rows, m output columns.

Sequential

Outputs also depend on stored state: the circuit has memory and feedback.

Lectures 10–16 are combinational; 18–22 sequential.

The analysis procedure

§4.3

- 1 Confirm it is combinational: no feedback, no storage.
- 2 Label every gate output that is a function of the inputs ($T_1, T_2, \dots$).
- 3 Write each label's Boolean function, working from the inputs toward the outputs.
- 4 Substitute until each output is in terms of the inputs only.
- 5 Build the truth table, one column per labelled signal; then say what the circuit does.

Two routes

Algebraic: substitute and simplify.

Tabular: evaluate every labelled signal on every row.

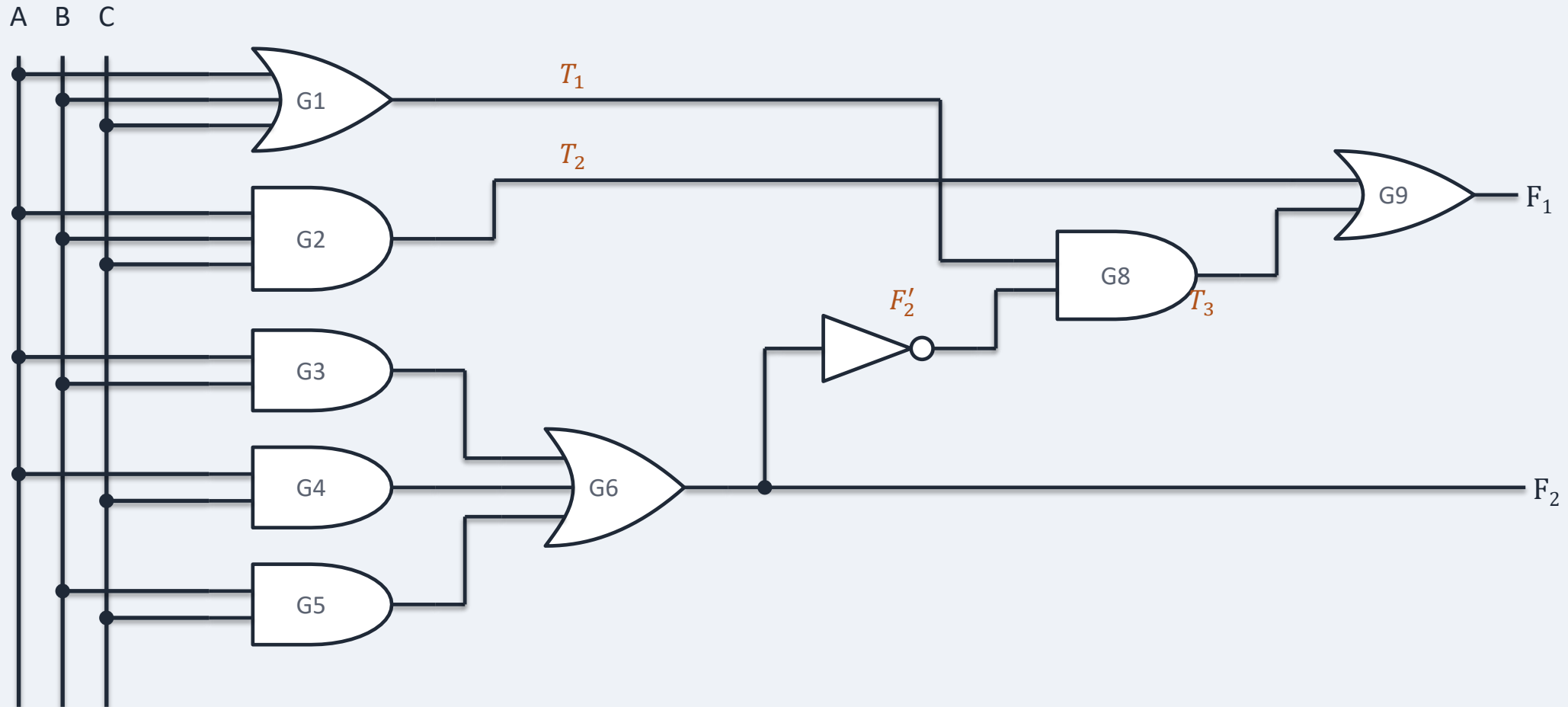
The table route is mechanical and hard to get wrong; use the algebra to check it.

Final step

Recognize the function: an adder, a comparator, a code converter...

Example 1 — Analyze this circuit

§4.3



Example 1 — Functions and table

§4.3

A B C	F_2	F_2'	T_1	T_2	T_3	F_1
0 0 0	0	1	0	0	0	0
0 0 1	0	1	1	0	1	1
0 1 0	0	1	1	0	1	1
0 1 1	1	0	1	0	0	0
1 0 0	0	1	1	0	1	1
1 0 1	1	0	1	0	0	0
1 1 0	1	0	1	0	0	0
1 1 1	1	0	1	1	0	1

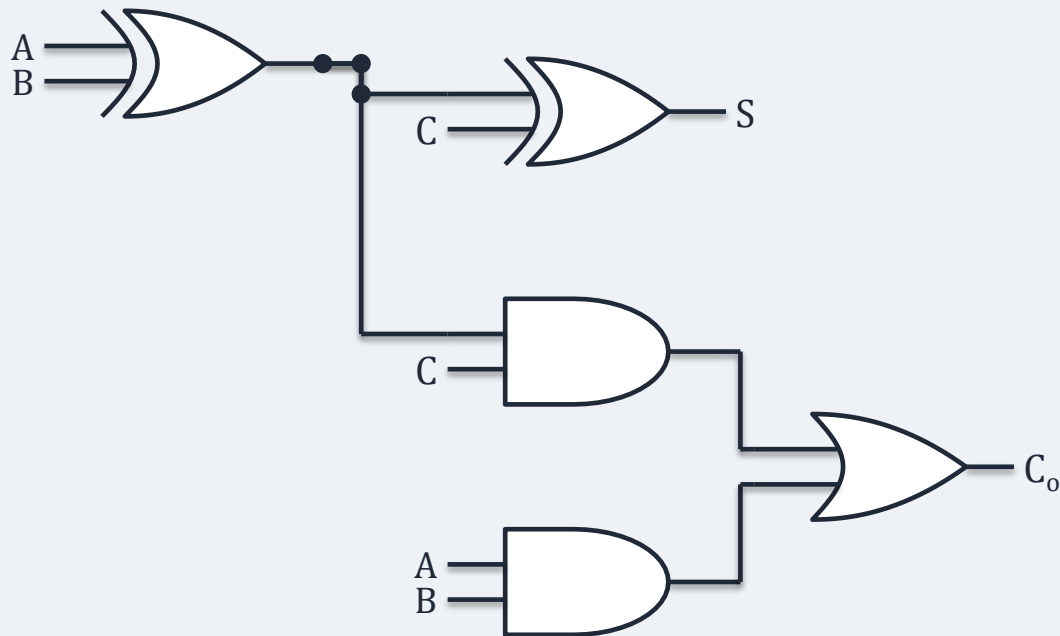
- 1 Label each gate output: $T_1 = A + B + C$ (G1), $T_2 = ABC$ (G2)
- 2 G3–G6: $F_2 = AB + AC + BC$
- 3 G7 inverts F_2 ; G8: $T_3 = T_1 F_2'$
- 4 G9: $F_1 = T_2 + T_3$
- 5 Fill the table one column at a time, left to right.
- 6 F_2 is 1 when two or more inputs are 1.
- 7 F_1 is 1 on 001, 010, 100, 111: an odd number of 1s
- 8 $F_1 = A \oplus B \oplus C = \Sigma m(1,2,4,7)$
- 9 **A full adder:** F_1 = sum, F_2 = carry

Your turn 1 — Analyze

§4.3

YOUR TURN

≈ 10 min



(a) Label the internal signals and write S and C_o in terms of A, B, C .

(b) Build the truth table.

(c) Compare it with Example 1. What does this circuit do?

Your turn 1 — Solution

§4.3

SOLUTION

A B C	$A \oplus B$	$(A \oplus B)C$	AB	S	C_o
0 0 0	0	0	0	0	0
0 0 1	0	0	0	1	0
0 1 0	1	0	0	1	0
0 1 1	1	1	0	0	1
1 0 0	1	0	0	1	0
1 0 1	1	1	0	0	1
1 1 0	0	0	1	0	1
1 1 1	0	0	1	1	1

- 1 First XOR: $T = A \oplus B$
- 2 Second XOR: $S = T \oplus C = A \oplus B \oplus C$
- 3 The two ANDs: $(A \oplus B)C$ and AB
- 4 The OR: $C_o = (A \oplus B)C + AB$
- 5 Fill the table column by column.
- 6 S matches F_1 and C_o matches F_2 of Example 1
- 7 **Also a full adder**, with 5 gates instead of 9

Break

10 minutes — we resume at the 0:50 mark

Next: the design procedure and code converters

The design procedure

§4.4

- 1 From the specification, fix the number of inputs and outputs and name them.
- 2 Build the truth table that defines each output.
- 3 Mark input combinations that cannot occur as don't-cares.
- 4 Simplify each output (K-map or tabular method).
- 5 Look for terms the outputs can share.
- 6 Draw the circuit; verify it against the table.

Constraints

Real designs weigh gates, gate inputs, propagation delay, packages and power.

A minimum for one output is not always the best choice when outputs share terms.

Verify

Evaluate the final circuit on at least two rows, or simulate it.

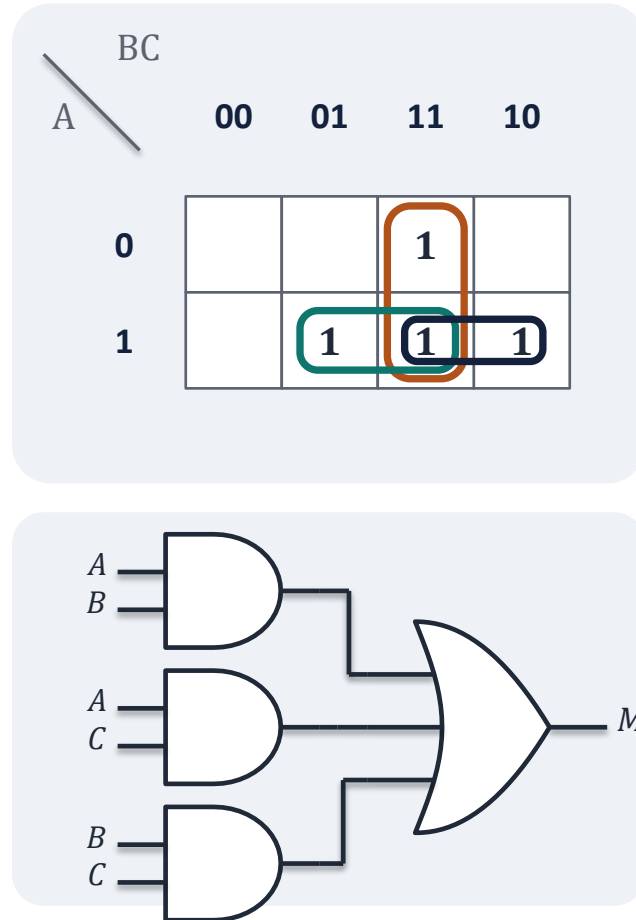
Example 2 — A majority circuit

§4.4

EXAMPLE

Three sensors A, B, C . Output $M = 1$ when a majority read 1. Design the minimum circuit.

A B C	M
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1



- 1 Inputs A, B, C ; one output M
- 2 $M = 1$ on rows with two or three 1s: 011, 101, 110, 111
- 3 So $M = \Sigma m(3, 5, 6, 7)$
- 4 Map: $m3, m7 \rightarrow BC$; $m5, m7 \rightarrow AC$; $m6, m7 \rightarrow AB$
- 5 Groups may overlap: $m7$ is used three times.
- 6
$$M = AB + AC + BC$$
- 7 NAND-NAND: 3 two-input NANDs + one 3-input NAND

Example 3 — BCD to excess-3: table

§4.4

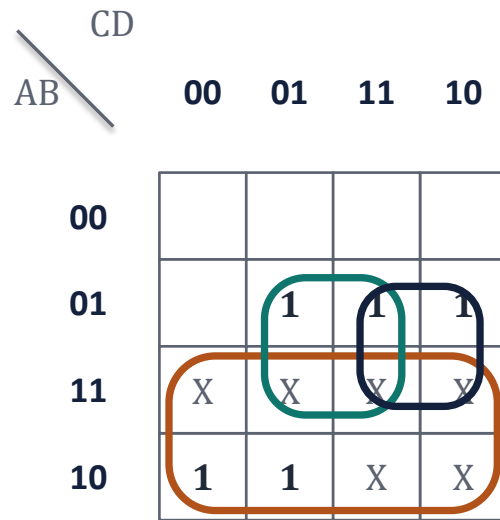
Dec	A B C D	w x y z
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Inputs 1010–1111: don't-care outputs

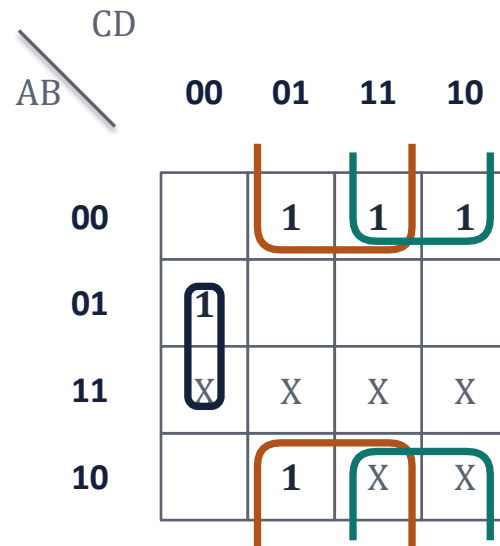
- 1 Input: one BCD digit $ABCD$ (A = MSB)
- 2 Output: excess-3 code $wxyz = \text{digit} + 3$
- 3 Only 0000–1001 occur: $d(10,11,12,13,14,15)$ for every output
- 4 Check: $5 + 3 = 8 \rightarrow 1000$; $9 + 3 = 12 \rightarrow 1100$ ✓
- 5 Next: one K-map per output

Example 3 — Four maps

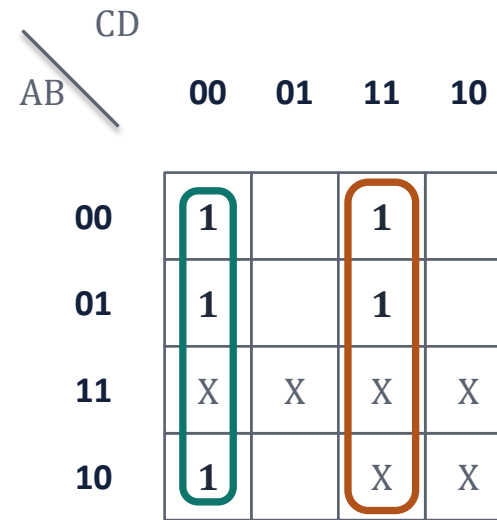
§4.4



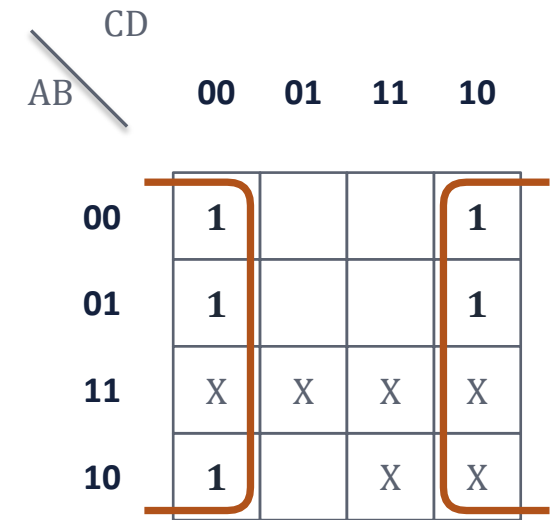
w



x



y



z

$$w = A + BC + BD \quad x = B'C + B'D + BC'D' \quad y = CD + C'D' \quad z = D'$$

Each is a minimum on its own map.

Example 3 — Reading the maps

§4.4

1 w : octet of rows 11, 10 (X as 1): $A = 1 \rightarrow A$

2 w : quad m5, m7, m13, m15: $B = 1, D = 1 \rightarrow BD$

3 w : quad m6, m7, m14, m15: $B = 1, C = 1 \rightarrow BC$

4 So $w = A + BC + BD$

5 x : quad m1, m3, m9, m11: $B = 0, D = 1 \rightarrow B'D$

6 x : quad m2, m3, m10, m11: $B = 0, C = 1 \rightarrow B'C$

7 x : pair m4, m12: $B = 1, C = 0, D = 0 \rightarrow BC'D'$

8 So $x = B'C + B'D + BC'D'$

9 y : column 11 $\rightarrow CD$; column 00 $\rightarrow C'D'$

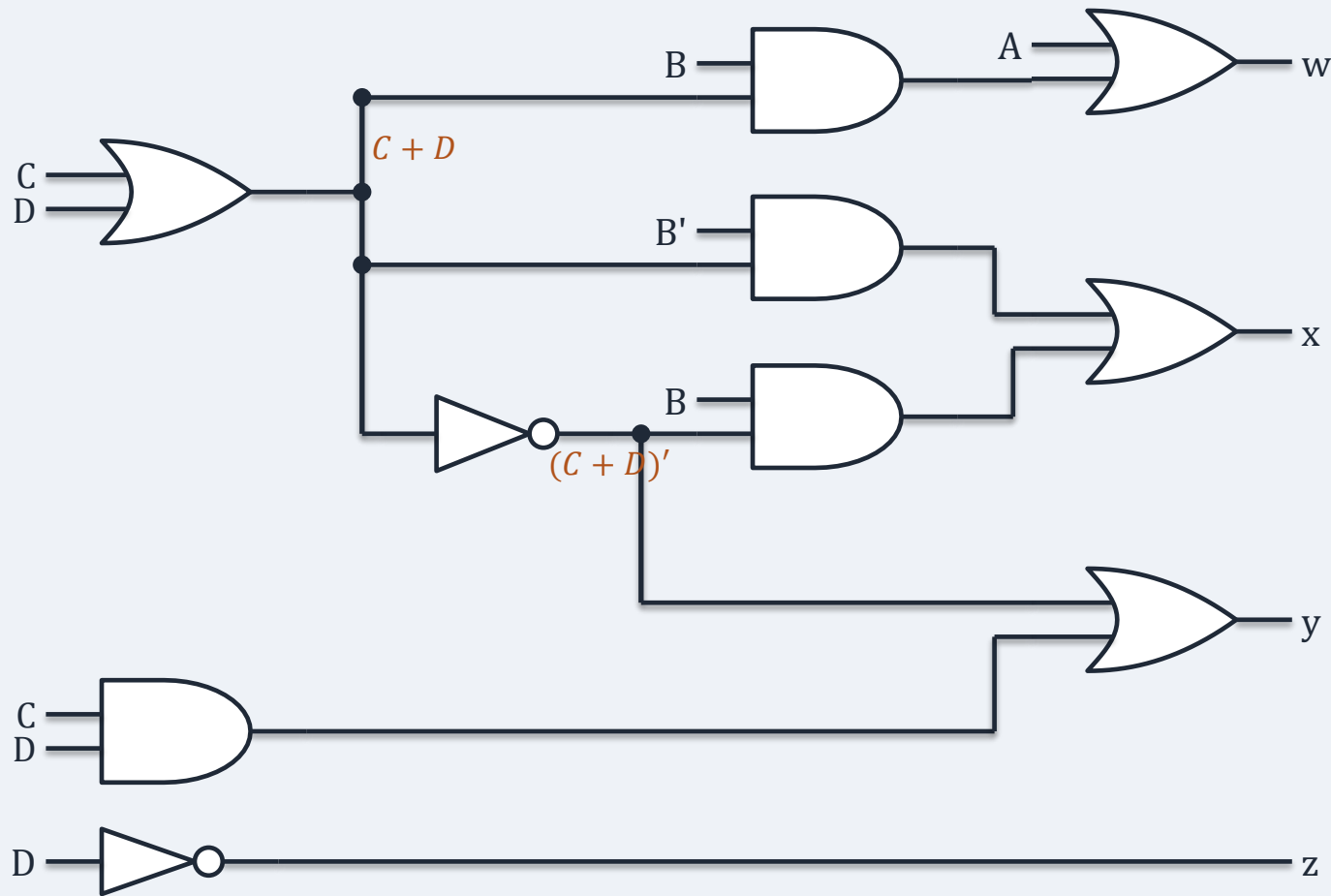
10 So $y = CD + C'D'$

11 z : the octet of columns 00 and 10: $D = 0$

12 So $z = D'$

Example 3 — Sharing $C + D$

§4.4



Factor for sharing

1. $w = A + BC + BD = A + B(C + D)$

2. $x = B'C + B'D + BC'D'$

$= B'(C + D) + BC'D'$

3. DeMorgan: $C'D' = (C + D)'$

$x = B'(C + D) + B(C + D)'$

4. $y = CD + C'D' = CD + (C + D)'$

5. $z = D'$

One $C + D$ gate and its complement feed three outputs.

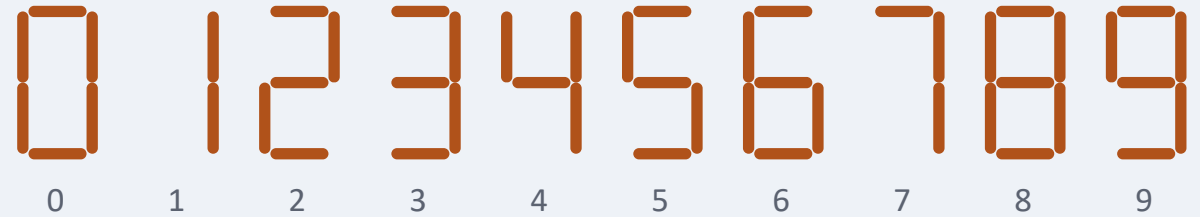
Example 4 — Seven-segment segment a

§4.4

EXAMPLE

A BCD digit $ABCD$ drives a seven-segment display. Design the output for segment a (active high).

CD \ AB	00	01	11	10
00	m0 1	m1	m3 1	m2 1
01	m4	m5 1	m7 1	m6 1
11	m12 X	m13 X	m15 X	m14 X
10	m8 1	m9 1	m11 X	m10 X



- 1 From the displays: a is off only for 1 and 4
- 2 $a = \Sigma m(0,2,3,5,6,7,8,9) + d(10, \dots, 15)$
- 3 Octet of rows 11, 10 (X as 1) $\rightarrow A$
- 4 Octet of columns 11, 10 $\rightarrow C$
- 5 Quad m5, m7, m13, m15 $\rightarrow BD$; four corners $\rightarrow B'D'$
- 6

$$a = A + C + BD + B'D'$$

Your turn 2 — Binary to Gray

§4.4

YOUR TURN

≈ 10 min

Design a circuit that converts a 3-bit binary number $B_2B_1B_0$ to its Gray code $G_2G_1G_0$.

- (a) Build the truth table.
- (b) Use a K-map for G_1 and for G_0 .
- (c) Draw the circuit with the fewest gates.

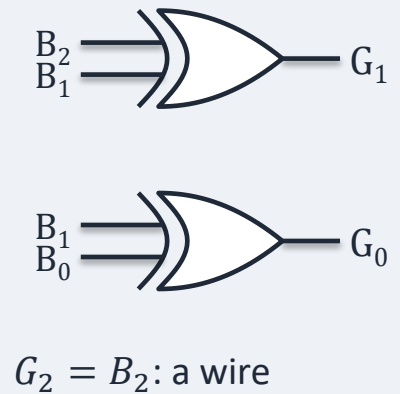
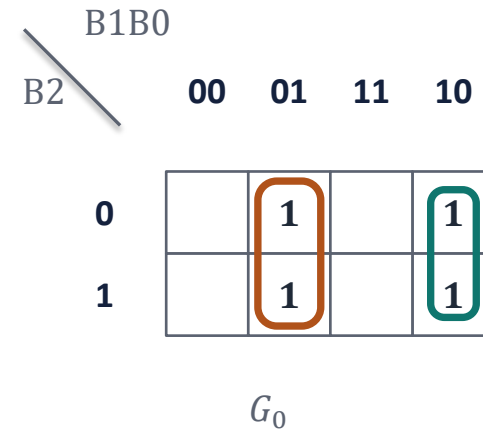
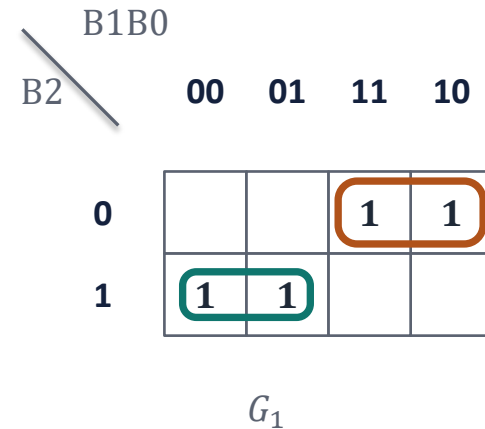
Hint: look for a checkerboard.

Your turn 2 — Solution

§4.4

SOLUTION

$B_2B_1B_0$	$G_2G_1G_0$
000	000
001	001
010	011
011	010
100	110
101	111
110	101
111	100



- 1 Table: $G_2 = B_2$, $G_1 = B_2 \oplus B_1$, $G_0 = B_1 \oplus B_0$
- 2 G_2 column equals B_2 : a wire
- 3 G_1 map: 1s at 2, 3, 4, 5
- 4 $m_2, m_3 \rightarrow B_2'B_1$; $m_4, m_5 \rightarrow B_2B_1'$
- 5 $G_1 = B_2'B_1 + B_2B_1' = B_2 \oplus B_1$
- 6 G_0 : $m_1, m_5 \rightarrow B_1'B_0$; $m_2, m_6 \rightarrow B_1B_0'$
- 7 $G_0 = B_1'B_0 + B_1B_0' = B_1 \oplus B_0$
- 8 Two XOR gates in all

Concept check

1

How can you tell from a diagram that a circuit is combinational?

2

Why label every internal gate output during analysis?

3

Why are inputs 1010–1111 don't-cares in a BCD converter?

4

When can a non-minimal form for one output give a cheaper circuit?

Key takeaways

Combinational

$$\text{outputs} = f(\text{inputs})$$

Analysis

label $\rightarrow$ substitute $\rightarrow$ table

Design

table $\rightarrow$ maps $\rightarrow$ share $\rightarrow$
draw

Majority

$$AB + AC + BC$$

Excess-3

$$z = D', y = CD + C'D'$$

Segment a

$$A + C + BD + B'D'$$

Analysis and design are the same table, read in opposite directions.

Homework #6 due at the start of Lecture 13

This afternoon's lab: Lab D — XOR gates, simplification and parity

Next: Exam 1, then decoders and encoders